МИНОБРНАУКИ РОССИИ

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ

ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра МоЭВМ

ОТЧЕТ

по лабораторной работе №1

Тема: Рекурсивный бэктрекинг. Исследование времени выполнения от размера квадрата.

Студент гр. 4343 _____ Овеси С.

Преподаватель _____ Курасов Н.С.

Санкт-Петербург

2023

Цель работы.

Исследовать рекурсивный бэктрекинг для задачи разбиения квадрата на минимальное количество меньших квадратов и изучить зависимость времени выполнения алгоритма от размера квадрата.

Основные теоретические положения.

Основные теоретические положения: задача разбиения квадрата на минимальное количество меньших квадратов является комбинаторной и решается методом рекурсивного бэктрекинга, при котором на каждом шаге выбирается квадрат максимального возможного размера для текущей пустой позиции, а пути, не способные улучшить текущее решение, отсеиваются.

Для чётных N существует известная оптимальная раскладка из 4 одинаковых квадратов, что позволяет избежать рекурсии. Время работы алгоритма и количество рекурсивных вызовов экспоненциально растут с увеличением N, обычно увеличиваясь, но с отдельными аномалиями, когда некоторые значения N требуют меньше шагов из-за особенностей раскладки.

N	Time (ms)	Steps
3	0	19
4	0	0
5	0	277
6	0	0
7	0	2290
8	0	0
9	0	1314
10	0	0
11	19	112559
12	0	0
13	35	333537
14	0	0
15	2	23434
16	0	0
17	545	4499156
18	0	0
19	2999	23282753
20	0	0
21	105	811461
22	0	0
23	15178	106382920
24	0	0
25	2211	11904224
26	0	0
27	3960	20467482
28	0	0

Рисунок 1.1 — Исследование времени выпольнения от размера квадрата

Из таблицы видно, что обычно, чем больше квадрат, тем больше времени и шагов занимает алгоритм, и это возрастает экспоненциально. Однако встречаются аномалии, например, N=21

и N=25, что типично для алгоритма с возвратом, поскольку оптимальное решение может быть найдено гораздо раньше, а остальные варианты отсеяны раньше.

Для всех N%2 == 0 нет необходимости вызывать алгоритм с возвратом, поскольку оптимальное решение — это 4 меньших квадрата одинакового размера.

Для малых N (здесь N<10) затраченное время пренебрежимо мало, поэтому время, указанное в миллисекундах, равно 0.

Алгоритм работает следующим образом: сначала выбирается квадрат в верхнем левом углу и принимается максимально возможный размер, равный N-1, и так рекурсивно слева направо, сверху вниз. После завершения этого процесса алгоритм переходит к следующему квадрату, пока не будут исчерпаны все возможности, после чего размер квадрата уменьшается на 1. Если количество квадратов в текущей ветви превышает количество квадратов в оптимальном решении, ветвь обрезается, алгоритм возвращается к последнему решению и пытается снова.

Выводы.

В ходе лабораторной работы был изучен метод рекурсивного бэктрекинга для разбиения квадрата на минимальное количество меньших квадратов. Реализация алгоритма показала, как с помощью полного перебора с возвратом и эвристик можно эффективно находить оптимальные решения для небольших размеров квадрата, а также как использование известных тривиальных раскладок для чётных N позволяет значительно сократить время вычислений.

Экспериментальное исследование зависимости времени работы и количества рекурсивных вызовов от размера квадрата продемонстрировало экспоненциальный рост вычислительной сложности с увеличением N, при этом наблюдаются отдельные аномалии, когда некоторые значения N требуют меньше шагов из-за особенностей размещения квадратов. Визуализация алгоритма показала последовательность размещения квадратов и позволила наглядно оценить работу бэктрекинга, что помогает лучше понять логику перебора и механизм отсечения неэффективных ветвей.

ПРОТОКОЛ

```
#include "raylib.h"
#include <chrono>
#include <climits>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <thread>
#include <vector>

using namespace std;

int N;
int best;
long long steps = 0;

struct Square {
  int x, y, w;
};

vector<Square> cur, ans;
int h[40];

const int WINDOW_SIZE = 600;
float cellSize;
bool visualize = false;

vector<Color> colors;
```

```
void drawSquares() {
  BeginDrawing();
  ClearBackground(RAYWHITE);

  // Draw grid
  for (int i = 0; i <= N; i++) {
    DrawLine(0, i * cellSize, WINDOW_SIZE, i * cellSize, GRAY);
    DrawLine(i * cellSize, 0, i * cellSize, WINDOW_SIZE, GRAY);
  }

  // Draw current squares
  for (size_t i = 0; i < cur.size(); i++) {
    Square s = cur[i];
    DrawRectangle(s.y * cellSize, s.x * cellSize, s.w * cellSize,
                  s.w * cellSize, colors[i]);
    DrawRectangleLines(s.y * cellSize, s.x * cellSize, s.w * cellSize,
                       s.w * cellSize, BLACK);
  }

  EndDrawing();
}

void solve(int used) {
  steps++; // count this recursive call

  if (used >= best)
    return;

  int maxEmpty = 0;
  for (int i = 0; i < N; i++)
    maxEmpty = max(maxEmpty, N - h[i]);
```

```
if (used + (maxEmpty + N - 1) / N >= best)
  return;

int minH = INT_MAX, pos = -1;
for (int i = 0; i < N; i++) {
  if (h[i] < minH) {
    minH = h[i];
    pos = i;
  }
}

if (minH == N) {
  best = used;
  ans = cur;
  return;
}

int maxW = 1;
while (pos + maxW < N && h[pos + maxW] == minH)
  maxW++;

int maxSize = min(N - minH, maxW);
if (used == 0)
  maxSize = N - 1;

for (int w = maxSize; w >= 1; w--) {
  for (int i = 0; i < w; i++)
    h[pos + i] += w;
  cur.push_back({minH, pos, w});

  if (visualize) {
    drawSquares();
```

```
      this_thread::sleep_for(chrono::milliseconds(50));
    }

    solve(used + 1);

    cur.pop_back();
    for (int i = 0; i < w; i++)
      h[pos + i] -= w;
  }
}

// Run solver for a specific N
void runSolver(int size, bool countOnly = false) {
  N = size;
  best = INT_MAX;
  cur.clear();
  ans.clear();
  for (int i = 0; i < N; i++)
    h[i] = 0;

  steps = 0; // reset BEFORE solve

  if (!countOnly) {
    colors.clear();
    for (int i = 0; i < N * N; i++)
      colors.push_back({(unsigned char)(rand() % 256),
                        (unsigned char)(rand() % 256),
                        (unsigned char)(rand() % 256), 255});
  }

  // Only run recursion for odd N
  if (N % 2 != 0)
```

```
    solve(0);
}

void printSolution() {
  cout << best << endl;
  for (auto &s : ans)
    cout << s.x + 1 << " " << s.y + 1 << " " << s.w << endl;
}

void runBenchmark() {
  cout << left << setw(10) << "N" << setw(20) << "Time (ms)" << setw(15)
      << "Steps" << endl;
  cout << string(45, '-') << endl;

  for (int size = 3; size <= 28; size++) {
    steps = 0;

    auto start = chrono::high_resolution_clock::now();
    runSolver(size, true); // countOnly = true
    auto end = chrono::high_resolution_clock::now();

    auto duration = chrono::duration_cast<chrono::milliseconds>(end - start);

    cout << left << setw(10) << size << setw(20) << duration.count() << setw(15)
        << steps << endl;
  }
}

int main(int argc, char *argv[]) {
  if (argc < 2) {
    cout << "Usage: " << argv[0] << " <N> | --benchmark | --visualizer <N>\n";
    return 1;
```

```
}

string arg = argv[1];

if (arg == "--benchmark") {
  runBenchmark();
} else if (arg == "--visualizer") {
  if (argc < 3) {
    cout << "Usage: " << argv[0] << " --visualizer <N>\n";
    return 1;
  }
  stringstream ss(argv[2]);
  int inputN;
  if (!(ss >> inputN)) {
    cout << "Invalid number: " << argv[2] << endl;
    return 1;
  }

  N = inputN;
  visualize = true;
  cellSize = (float)WINDOW_SIZE / N;

  InitWindow(WINDOW_SIZE, WINDOW_SIZE, "Square Tiling Visualizer");
  SetTargetFPS(60);

  if (N % 2 == 0) {
    best = 4;
    ans = {{0, 0, N / 2},
           {0, N / 2, N / 2},
           {N / 2, 0, N / 2},
           {N / 2, N / 2, N / 2}};
```

```
    colors.clear();
    for (size_t i = 0; i < ans.size(); i++)
      colors.push_back({(unsigned char)(rand() % 256),
                        (unsigned char)(rand() % 256),
                        (unsigned char)(rand() % 256), 255});
  } else {
    runSolver(N);
  }

  cur = ans;
  while (!WindowShouldClose()) {
    drawSquares();
  }
  CloseWindow();

} else {
  stringstream ss(arg);
  int inputN;
  if (!(ss >> inputN)) {
    cout << "Invalid number: " << arg << endl;
    return 1;
  }

  N = inputN;
  cout << N << endl;

  if (N % 2 == 0) {
    int k = N / 2;
    cout << 4 << endl;
    cout << 1 << " " << 1 << " " << k << endl;
    cout << 1 << " " << k + 1 << " " << k << endl;
    cout << k + 1 << " " << 1 << " " << k << endl;
```

```
      cout << k + 1 << " " << k + 1 << " " << k << endl;
    } else {
      runSolver(N);
      printSolution();
    }
  }

  return 0;
}
```